

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics Practical

Practical – III

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 14/08/2026	Date/Time of Submission: 14/08/2026

AIM: Finding Relationships Among Variables:

A. Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load Dataset
df = pd.read_csv("housing.csv")
# Display First Five Records
print(df.head())
# Display Numerical Columns
print(df.select_dtypes(include="number").columns)
```

```
...   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0      -122.23    37.88           41.0           880.0           129.0
1      -122.22    37.86           21.0          7099.0          1106.0
2      -122.24    37.85           52.0          1467.0           190.0
3      -122.25    37.85           52.0          1274.0           235.0
4      -122.25    37.85           52.0          1627.0           280.0

   population  households  median_income  median_house_value  ocean_proximity
0         322.0        126.0         8.3252         452600.0        NEAR BAY
1        2401.0       1138.0         8.3014        358500.0        NEAR BAY
2         496.0        177.0         7.2574        352100.0        NEAR BAY
3         558.0        219.0         5.6431        341300.0        NEAR BAY
4         565.0        259.0         3.8462        342200.0        NEAR BAY
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='object')
```

```
# Calculate Correlation Matrix
correlation = df.corr(numeric_only=True)
print(correlation)
```

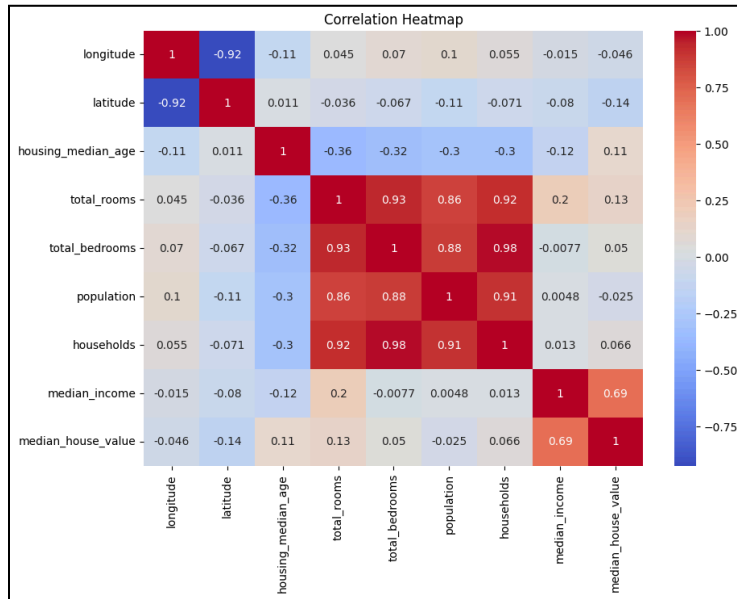
```
# Display Correlation Heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()
```

OUTPUT:

```
...   longitude  longitude  latitude  housing_median_age  total_rooms  \
longitude  1.000000  -0.924664  -0.108197  -0.044568
latitude   -0.924664  1.000000   0.011173  -0.036100
housing_median_age  -0.108197  0.011173  1.000000  -0.361262
total_rooms    0.044568  -0.036100  -0.361262  1.000000
total_bedrooms  0.069608  -0.066983  -0.320451  0.930380
population     0.099773  -0.108785  -0.296244  0.857126
households     0.055310  -0.071035  -0.302916  0.918484
median_income  -0.015176  -0.079809  -0.119034  0.198050
median_house_value -0.045967  -0.144160  0.105623  0.134153

   total_bedrooms  population  households  median_income  \
longitude    0.069608    0.099773    0.055310  -0.015176
latitude    -0.066983    -0.108785    -0.071035  -0.079809
housing_median_age  -0.320451  -0.296244  -0.302916  -0.119034
total_rooms    0.930380    0.857126    0.918484    0.198050
total_bedrooms  1.000000    0.877747    0.979728  -0.007723
population    0.877747    1.000000    0.907222  0.004834
households    0.979728    0.907222    1.000000  0.013033
median_income  -0.007723  0.004834  0.013033  1.000000
median_house_value  0.049686  -0.024650  0.065843  0.688075

   median_house_value
longitude    -0.045967
latitude     -0.144160
housing_median_age  0.105623
total_rooms    0.134153
total_bedrooms  0.049686
population    -0.024650
households     0.065843
median_income   0.688075
median_house_value  1.000000
```



B) Apply regression analysis to identify the relationship between an independent variable (e.g., advertising expenditure) and a dependent variable (e.g., sales revenue).

CODE:

```
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

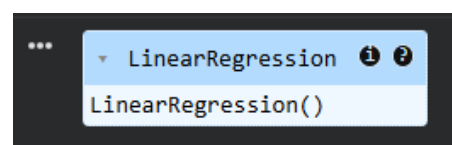
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load Dataset
df = pd.read_csv("housing.csv")

# Select Independent and Dependent Variables
X = df[["median_income"]]
y = df["median_house_value"]

# Split Dataset into Training and Testing Data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train the Model
```



```
model.fit(X_train, y_train)
```

```
# Display Coefficient and Intercept
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
# Predict House Values
y_pred = model.predict(X_test)
# Display First 10 Predictions
print("Predicted Values:")
print(y_pred[:10])
```

```
... Predicted Values:
[114958.91676996 150606.88213964 190393.71844449 285059.38345102
 200663.31816103 242165.24890609 257647.22610228 199229.18051176
 245893.1681172 384677.43607096]
```

```
# Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

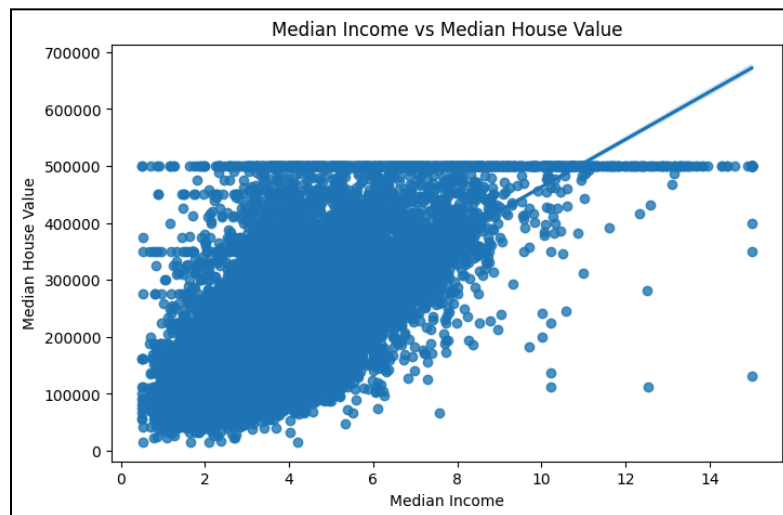
```
... Mean Squared Error: 7091157771.76555
```

```
# Calculate R2 Score
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

```
... R2 Score: 0.45885918903846656
```

```
# Scatter Plot with Regression Line
plt.figure(figsize=(8, 5))
sns.regplot(
    x="median_income",
    y="median_house_value",
    data=df
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median Income vs Median House Value")
plt.show()
```

OUTPUT:



1. Calculate the complete correlation matrix for all numerical variables.

Identify:

- The strongest positive correlation.
- The strongest negative correlation.
- The weakest correlation.
- Display the results.

CODE:

```
import pandas as pd
import numpy as np
df = pd.read_csv("housing.csv")
# Select numerical variables
numerical_data = df.select_dtypes(include=np.number)
# Calculate correlation matrix
correlation_matrix = numerical_data.corr()
print("Complete Correlation Matrix:")
print(correlation_matrix)
# Get correlation pairs
pairs = []
columns = correlation_matrix.columns
for i in range(len(columns)):
    for j in range(i + 1, len(columns)):
        pairs.append(
            (columns[i], columns[j], correlation_matrix.iloc[i, j])
        )
# Strongest positive correlation
strongest_positive = max(pairs, key=lambda x: x[2])
# Strongest negative correlation
strongest_negative = min(pairs, key=lambda x: x[2])
# Weakest correlation
weakest = min(pairs, key=lambda x: abs(x[2]))
print("\nStrongest Positive Correlation:")
print(strongest_positive)
print("\nStrongest Negative Correlation:")
print(strongest_negative)
print("\nWeakest Correlation:")
print(weakest)
```

OUTPUT:

```
Strongest Positive Correlation:
('total_bedrooms', 'households', np.float64(0.9797282708045709))

Strongest Negative Correlation:
('longitude', 'latitude', np.float64(-0.9246644339150368))

Weakest Correlation:
('population', 'median_income', np.float64(0.004834345627653126))
```

```
Complete Correlation Matrix:
'''
longitude    longitude    latitude    housing_median_age    total_rooms    \
longitude      1.000000    -0.924664    -0.108197    0.044568
latitude      -0.924664    1.000000     0.011173    -0.036109
housing_median_age -0.108197    0.011173    1.000000    -0.361262
total_rooms     0.044568    -0.036109    -0.361262    1.000000
total_bedrooms  0.069686    -0.066983    -0.320451    0.930380
population     0.099773    -0.108785    -0.296244    0.857126
households     0.055310    -0.071035    -0.302916    0.918484
median_income  -0.015176    -0.079809    -0.119034    0.198050
median_house_value -0.045967    -0.144160    0.105623    0.134153

longitude    total_bedrooms    population    households    median_income    \
longitude      0.069686    0.099773    0.055310    -0.015176
latitude      -0.066983    -0.108785    -0.071035    -0.079809
housing_median_age -0.320451    -0.296244    -0.302916    -0.119034
total_rooms     0.930380    0.857126    0.918484    0.198050
total_bedrooms  1.000000    0.877747    0.979728    -0.007723
population     0.877747    1.000000    0.907222    0.004834
households     0.979728    0.907222    1.000000    0.013033
median_income  -0.007723    0.004834    0.013033    1.000000
median_house_value 0.049686    -0.024650    0.065843    0.688075

longitude    median_house_value
longitude      -0.045967
latitude      -0.144160
housing_median_age 0.105623
total_rooms     0.134153
total_bedrooms  0.049686
population     -0.024650
households     0.065843
median_income  0.688075
median_house_value 1.000000
```

Strongest Positive Correlation:

total_bedrooms and households

Correlation = 0.9797

Strongest Negative Correlation:

longitude and latitude

Correlation = -0.9247

Weakest Correlation:

population and median_income

Correlation = 0.0048

2)Build a regression model using:

X =median_income

X=housing_median_age

X=total_rooms

X=population

X=households

Y =median_house_value

Evaluate the model using:

- R2
- Mean Squared Error

Compare this model with the simple regression model using only median_income.

CODE:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
df = pd.read_csv("housing.csv")
# Independent variables
X = df[
    [
        "median_income",
        "housing_median_age",
        "total_rooms",
        "population",
        "households"
    ]
]
# Dependent variable
```

```

y = df["median_house_value"]
# Multiple Linear Regression
model = LinearRegression()
model.fit(X, y)
# Predictions
y_pred = model.predict(X)
# Evaluation
r2 = r2_score(y, y_pred)
mse = mean_squared_error(y, y_pred)
print("Multiple Linear Regression")
print("-----")
print("Intercept:", model.intercept_)
print("\nCoefficients:")
for feature, coefficient in zip(X.columns, model.coef_):
    print(feature, ":", coefficient)
print("\nR2 Score:", r2)
print("Mean Squared Error:", mse)

```

OUTPUT:

```

... Multiple Linear Regression
-----
Intercept: -37926.60615574694

Coefficients:
median_income : 45969.659381936064
housing_median_age : 1842.1494875739586
total_rooms : -13.946204039688382
population : -39.44546486713351
households : 214.35386822082057

R2 Score: 0.5629054644856253
Mean Squared Error: 5820133599.282842

```

3)Find the correlation between:

- **total_rooms**
- **median_house_value**

CODE:

```

# Simple Linear Regression
X_simple = df[["median_income"]]
simple_model = LinearRegression()
simple_model.fit(X_simple, y)
# Predictions
y_pred_simple = simple_model.predict(X_simple)
# Evaluation
r2_simple = r2_score(y, y_pred_simple)
mse_simple = mean_squared_error(y, y_pred_simple)
print("Simple Linear Regression")
print("-----")
print("R2 Score:", r2_simple)
print("Mean Squared Error:", mse_simple)

```

OUTPUT:

```

... Simple Linear Regression
-----
R2 Score: 0.47344749180719903
Mean Squared Error: 7011311502.929527

```